



ALLIANCE
UNIVERSITY
Private University established in Karnataka State by Act No. 84 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi.

NAAC
GRADE A+
ACCREDITED UNIVERSITY

Bachelor of Computer
Applications Semester – V
Deep
Learning
Experiment
No.: 5

Title: ACTIVATION FUNCTIONS USING PYTHON

Submitted By:

Name:

Name:

Shashank

M

Reg.

No.:2411021

240049Date:

30-08-2026

Submitted To:
Faculty Name: Nagasri p

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru – 562106

PROJECT-5
ACTIVATION FUNCTIONS USING PYTHON
Subject: Deep Learning

1. Introduction

Deep Learning is a branch of Artificial Intelligence that uses artificial neural networks to learn patterns from large amounts of data. Neural networks are inspired by the basic working principle of the human brain, where interconnected neurons process information and produce an output.

A neural network consists of several important components, including input layers, hidden layers, weights, biases, and activation functions. Among these components, **activation functions play a very important role because they introduce non-linearity into the network.**

A neural network without an activation function would essentially perform only linear mathematical operations. Even if several layers were added, the overall network would still behave like a linear model. This would make it difficult for the network to learn complex relationships present in real-world data.

Activation functions solve this problem by transforming the output of neurons in a non-linear way.

For example, consider a neuron receiving several inputs:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

where:

- $x_1, x_2, \dots, x_n$ are input values
- $w_1, w_2, \dots, w_n$ are weights
- b is the bias
- z is the weighted sum

The activation function is then applied:

$$a = f(z)$$

where f represents the activation function.

In this project, three commonly used activation functions are studied and implemented using Python:

1. **Sigmoid**
2. **ReLU**

3. Tanh

The project also compares their outputs and visualizes their behavior using graphs.

2. Project Objective

The main objectives of this project are:

- To understand the concept of activation functions in Deep Learning.
 - To understand why activation functions are required in neural networks.
 - To implement Sigmoid, ReLU, and Tanh using Python.
 - To calculate the output of each activation function for different input values.
 - To visualize the behavior of different activation functions.
 - To compare the advantages and disadvantages of Sigmoid, ReLU, and Tanh.
 - To understand where different activation functions are commonly used.
 - To gain practical experience with Python libraries such as NumPy and Matplotlib.
-

3. What is an Activation Function?

An activation function is a mathematical function applied to the output of a neuron.

The basic structure of a neuron can be represented as:

$$z = \sum_{i=1}^n w_i x_i + b$$

The activation function then transforms this value:

$$y = f(z)$$

The transformed value is passed to the next neuron or layer.

Why is it necessary?

The most important reason for using activation functions is to introduce **non-linearity**.

Real-world problems are rarely completely linear. For example:

- Image recognition
- Speech recognition
- Fraud detection
- Medical diagnosis
- Natural language processing
- Object detection

all involve complex relationships.

Activation functions allow neural networks to model these complex relationships.

4. Importance of Non-Linearity

Suppose a neural network contains several layers but does not use activation functions.

A simple neuron can be represented as:

$$y = wx + b$$

If another layer is added:

$$y_2 = w_2(w_1x + b_1) + b_2$$

This can still be simplified into another linear equation:

$$y_2 = wx + b$$

Therefore, simply adding more layers does not necessarily make the network capable of learning complex patterns.

Activation functions break this limitation.

For example:

$$y = f(wx + b)$$

where f is a non-linear activation function.

This allows the network to learn complicated relationships between inputs and outputs.

5. Types of Activation Functions

There are many activation functions used in neural networks. Some commonly known functions include:

- Sigmoid
- ReLU
- Tanh
- Leaky ReLU
- Softmax
- ELU
- GELU

This project focuses on three fundamental activation functions:

5.1 Sigmoid

5.2 ReLU

5.3 Tanh

6. Sigmoid Activation Function

The Sigmoid activation function is one of the oldest and most commonly studied activation functions.

Its mathematical formula is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The output of Sigmoid is always between **0 and 1**.

$$0 < \sigma(x) < 1$$

Example

For:

$$x = 0$$

we get:

$$\begin{aligned}\sigma(0) &= \frac{1}{1 + e^0} \\ &= \frac{1}{1 + 1} \\ &= 0.5\end{aligned}$$

Therefore, the Sigmoid output for zero is **0.5**.

Behavior

For a large positive input, the output approaches 1.

For a large negative input, the output approaches 0.

For example:

Input Sigmoid Output

-5 0.0067

Input Sigmoid Output

-2	0.1192
-1	0.2689
0	0.5000
1	0.7311
2	0.8808
5	0.9933

Advantages

- Simple mathematical function.
- Output is restricted between 0 and 1.
- Useful when an output needs to represent a probability.
- Commonly associated with binary classification output layers.

Disadvantages

The major disadvantage is the **vanishing gradient problem**.

When the input becomes very large or very negative, the Sigmoid function becomes almost flat. Its gradient becomes very small, which can make learning slow in deep networks.

7. ReLU Activation Function

ReLU stands for **Rectified Linear Unit**.

It is one of the most widely used activation functions in modern neural networks.

The formula is:

$$ReLU(x) = \max(0, x)$$

This means:

$$ReLU(x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0 \end{cases}$$

Example 1

If:

$$x = -5$$

then:

$$ReLU(-5) = 0$$

Example 2

If:

$$x = 5$$

then:

$$ReLU(5) = 5$$

Therefore, negative values are converted to zero while positive values remain unchanged.

Sample Output

Input ReLU Output

-5	0
-2	0
-1	0

Input ReLU Output

0	0
1	1
2	2
5	5

Advantages

- Very simple to calculate.
- Computationally efficient.
- Helps reduce the vanishing-gradient problem for positive inputs.
- Commonly used in hidden layers of neural networks.
- Produces sparse activations because negative inputs become zero.

Disadvantage

One important problem is called the **dying ReLU problem**.

If a neuron consistently receives negative inputs, its output becomes zero. Under some training conditions, that neuron may stop contributing useful gradients and effectively become inactive.

8. Tanh Activation Function

Tanh stands for **Hyperbolic Tangent**.

Its formula is:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The output range is:

$$-1 < \tanh(x) < 1$$

Unlike Sigmoid, Tanh is centered around zero.

Example

For:

$$x = 0$$

we get:

$$\tanh(0) = 0$$

For a large positive value, Tanh approaches 1.

For a large negative value, Tanh approaches -1.

Sample Output

Input Tanh Output

-5	-1.0000
-2	-0.9640
-1	-0.7616
0	0.0000
1	0.7616
2	0.9640
5	0.9999

Advantages

- Output is centered around zero.

- Produces both positive and negative values.
- Stronger gradients around zero compared with Sigmoid.
- Can be useful in some neural-network architectures.

Disadvantages

Like Sigmoid, Tanh can suffer from the **vanishing gradient problem** when the input becomes very large or very negative.

9. Comparison of Sigmoid, ReLU and Tanh

Feature	Sigmoid	ReLU	Tanh
Output Range	0 to 1	0 to ∞	-1 to 1
Zero Centered	No	No	Yes
Negative Inputs	Small positive output	0	Negative output
Positive Inputs	Between 0 and 1	Same as input	Between 0 and 1
Computational Cost	Moderate	Low	Moderate
Vanishing Gradient	Yes	Less for positive values	Yes
Common Usage	Binary output	Hidden layers	Hidden layers / some sequence models

10. Python Implementation

Python provides several libraries that make numerical calculations and visualization easy. For this project, two important libraries are used:

NumPy

NumPy is used for numerical calculations and creating arrays of input values.

Matplotlib

Matplotlib is used to visualize the activation functions.

The complete Python program is:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Sigmoid activation function
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

```
# ReLU activation function
```

```
def relu(x):
    return np.maximum(0, x)
```

```
# Tanh activation function
```

```
def tanh(x):
    return np.tanh(x)
```

```
# Generate input values
```

```
x = np.linspace(-5, 5, 100)
```

```
# Calculate outputs
```

```

sigmoid_output = sigmoid(x)
relu_output = relu(x)
tanh_output = tanh(x)

# Sample input values
sample_values = [-5, -2, -1, 0, 1, 2, 5]

print("Input\tSigmoid\tReLU\tTanh")

for value in sample_values:
    print(
        value,
        "\t",
        round(sigmoid(value), 4),
        "\t\t",
        round(relu(value), 4),
        "\t\t",
        round(tanh(value), 4)
    )

# Plot activation functions
plt.figure(figsize=(10, 6))

plt.plot(x, sigmoid_output, label="Sigmoid")
plt.plot(x, relu_output, label="ReLU")
plt.plot(x, tanh_output, label="Tanh")

plt.title("Comparison of Activation Functions")
plt.xlabel("Input")
plt.ylabel("Output")

plt.axhline(0, linewidth=0.8)
plt.axvline(0, linewidth=0.8)

plt.grid(True)
plt.legend()

plt.show()

```

11. Explanation of the Code

Importing NumPy

import numpy as np

NumPy is imported to perform mathematical operations and generate numerical arrays.

Importing Matplotlib

import matplotlib.pyplot as plt

Matplotlib is used to create graphs.

Defining Sigmoid

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

This function calculates the Sigmoid value for the given input.

Defining ReLU

```
def relu(x):  
    return np.maximum(0, x)
```

This compares the input with zero and returns whichever value is larger.

Defining Tanh

```
def tanh(x):  
    return np.tanh(x)
```

NumPy's built-in `tanh()` function calculates the hyperbolic tangent.

Generating Inputs

```
x = np.linspace(-5, 5, 100)
```

This generates 100 equally spaced values between -5 and 5.

Calculating Outputs

```
sigmoid_output = sigmoid(x)  
relu_output = relu(x)  
tanh_output = tanh(x)
```

Each input value is passed through all three activation functions.

Plotting

```
plt.plot(x, sigmoid_output)  
plt.plot(x, relu_output)  
plt.plot(x, tanh_output)
```

These commands plot the three functions so their behaviors can be compared visually.

12. Output Analysis

The program generates output values for different inputs.

For example, when the input is **-5**:

- Sigmoid ≈ 0.0067
- ReLU = 0
- Tanh ≈ -1

When the input is **0**:

- Sigmoid = 0.5
- ReLU = 0
- Tanh = 0

When the input is **5**:

- Sigmoid ≈ 0.9933
- ReLU = 5
- Tanh ≈ 0.9999

This demonstrates that the three functions transform the same input in completely different ways.

13. Visualization and Interpretation

The graph provides an easy way to understand the differences between the activation functions.

Sigmoid Curve

The Sigmoid curve has an **S-shaped structure**.

It starts close to zero for negative values and gradually increases toward one for positive values.

ReLU Curve

The ReLU graph is zero for all negative inputs.

After zero, it increases linearly.

This gives ReLU a very simple shape compared with Sigmoid and Tanh.

Tanh Curve

Tanh also has an S-shaped curve, but unlike Sigmoid, it ranges from approximately -1 to 1. It is therefore **zero-centered**.

14. Practical Applications

Activation functions are used in many Deep Learning applications.

Image Classification

Activation functions are used inside neural networks that classify images into categories.

For example:

- Cat
- Dog
- Car
- Person

ReLU is commonly used in hidden layers of deep neural networks.

Binary Classification

Sigmoid can be used in the output layer when predicting one of two classes.

For example:

0 = Not Fraud
1 = Fraud

The Sigmoid output can be interpreted as a probability-like score.

Natural Language Processing

Activation functions are also used in neural networks that process text and language.

Tanh has historically been used in recurrent neural network architectures and related components.

Fraud Detection

A neural network can learn patterns associated with fraudulent transactions. Activation functions allow the network to model non-linear relationships among transaction features.

15. Advantages of Using Activation Functions

Activation functions provide several important benefits:

1. Non-Linearity

They allow neural networks to learn complicated relationships.

2. Better Learning

They allow multiple neural-network layers to learn different patterns.

3. Output Transformation

They can restrict or transform outputs into useful ranges.

4. Improved Representation

They help neural networks build increasingly complex representations of input data.

5. Problem-Specific Selection

Different activation functions can be selected based on the requirements of a particular network.

16. Limitations

Activation functions are useful, but no single activation function is perfect.

Sigmoid

Main problems:

- Vanishing gradients
- Not zero-centered
- Can slow learning in deep networks

ReLU

Main problem:

- Dying ReLU

Negative inputs produce zero, which can cause some neurons to become inactive.

Tanh

Main problems:

- Vanishing gradients for extreme inputs
- More computationally expensive than ReLU

Therefore, choosing an activation function depends on the architecture and problem.

17. Result

The Sigmoid, ReLU, and Tanh activation functions were successfully implemented using Python.

The experiment demonstrated that:

- **Sigmoid** converts inputs into values between 0 and 1.
- **ReLU** converts negative inputs to zero and keeps positive inputs unchanged.
- **Tanh** converts inputs into values between -1 and 1.
- Each function has different mathematical behavior.
- Visualization makes the differences between the functions easier to understand.

The Python program successfully generated numerical outputs and graphical representations of all three functions.

18. Conclusion

Activation functions are a fundamental component of Deep Learning and artificial neural networks. They introduce non-linearity, allowing neural networks to learn complex patterns that cannot be represented using simple linear equations.

In this project, three important activation functions—**Sigmoid, ReLU, and Tanh**—were studied and implemented using Python.

The Sigmoid function was found to be useful for producing probability-like outputs between 0 and 1. ReLU was observed to be computationally simple and particularly useful in hidden layers. Tanh was found to provide zero-centered outputs ranging from -1 to 1.

The practical implementation using NumPy and Matplotlib helped demonstrate how mathematical equations can be converted into working Python programs. The graphical comparison also made it easier to understand how each activation function responds to positive, negative, and zero input values.

Therefore, activation functions are not merely mathematical formulas; they are essential components that allow modern neural networks to learn meaningful and complex patterns from data.

19. Personal Observation

While implementing these functions, I found that visualizing the mathematical formulas made the concept much easier to understand than studying the equations alone.

The most noticeable difference was between ReLU and the other two functions. ReLU is very simple because it completely removes negative values, while Sigmoid and Tanh smoothly transform the input.

This experiment also showed that there is no single activation function that is best for every situation. The appropriate function depends on the network architecture and the type of problem being solved.

20. References

1. Deep Learning concepts and neural network fundamentals.
2. Python NumPy documentation.
3. Python Matplotlib documentation.
4. Standard mathematical definitions of Sigmoid, ReLU, and Tanh activation functions.